# Programação Orientada a Objetos

## Parte 2

Herança, Polimorfismo, Abstratas, Interfaces

Prof. Geovani | 10/03/2026 - Aula 5

# Marcos Importantes

| Data | Evento | Detalhes |
|---|---|---|
| 03/02 | ■ Aula 1 | Apresentação (Realizada) |
| 17/03 | ■ Entrega 1 | Proposta do Projeto |
| 05/05 | ■ AP (30%) | Avaliação Parcial |
| 26/05 | ■ Entrega 2 | Modelagem do BD |
| 02/06 | ■ Aula 16 | JWT - ÚLTIMA AULA |
| 09/06 | ■ AF (30%) | APRESENTAÇÃO FINAL |
| 16/06 | ■ AI (40%) | Avaliação Interdisciplinar |
| 23/06 | ■ AIS | Substitutiva (se aplicável) |
| 30/06 | ■ Aula 20 | Encerramento |

# Atenção ao nosso cronograma

| Aula | Data | Conteúdo Principal | Evento/Entrega |
|---|---|---|---|
| 1 | 03/02 | Apresentação da Disciplina | |
| 2 | 10/02 | Fundamentos C# - Parte 1 | - |
| - | 17/02 | CARNAVAL - Sem aula | Feriado |
| 3 | 24/02 | Fundamentos C# Parte 2 + Coleções | - |
| ~~4~~ | ~~03/03~~ | ~~OOP - Parte 1~~ | ~~-~~ |
| 5 | 10/03 | OOP - Parte 2 | - |
| 6 | 17/03 | .NET Core e ASP.NET Core | Entrega Proposta de Projeto |
| 7 | 24/03 | SOLID + Design Patterns | - |
| 8 | 31/03 | Clean Architecture | - |
| - | 03-04/04 | Paixão de Cristo - Suspensão | Feriado |
| 9 | 07/04 | HTTP e REST | - |
| 10 | 14/04 | Construindo APIs REST | - |
| - | 21/04 | Tiradentes - Feriado | Feriado |
| 11 | 28/04 | Middleware e Filtros | - |
| 12 | 05/05 | AVALIAÇÃO PARCIAL (AP) | AP - 30% |
| 13 | 12/05 | PostgreSQL + EF Core | - |
| 14 | 19/05 | Repository Pattern + BD | - |
| 15 | 26/05 | Queries Avançadas | Entrega da Arquitetura de Software do projeto |
| 16 | **02/06** | **AUTENTICAÇÃO JWT** | **FIM CONTEÚDO** |
| - | 04-06/06 | Corpus Christi - Suspensão | Feriado |
| 17 | 09/06 | APRESENTAÇÃO FINAL (AF) | AF - 30% |
| 18 | 16/06 | AVALIAÇÃO INTERDISCIPLINAR (AI) | AI - 40% |
| 19 | 23/06 | Avaliação Substitutiva (AIS) | AIS |
| 20 | 30/06 | Encerramento + Revisão | Fim Semestre |

# Objetivos de Hoje

🎯 Dominar Herança (Inheritance)
🎯 Implementar Polimorfismo
🎯 Criar Classes Abstratas
🎯 Trabalhar com Interfaces
🎯 Entender virtual, override, abstract
🎯 Praticar com hierarquias de classes
🎯 Preview dos princípios SOLID

# Revisão: Aula 4 - OOP Parte 1

- ✅ Paradigma OOP e 4 Pilares
- ✅ Classes e Objetos
- ✅ Fields vs Properties
- ✅ Construtores e Sobrecarga
- ✅ Encapsulamento (private, public)
- ✅ Modificadores de Acesso
- ✅ Métodos (instância e static)
- ✅ this keyword

# Herança

Reutilizando Código

# Herança (Inheritance) - Conceito

📌 Definição:
Mecanismo onde uma classe HERDA características (atributos e métodos) de outra classe

🎯 Objetivo:
- Reutilizar código
- Criar hierarquias (classificações)
- Evitar duplicação
- Estabelecer relação "É UM(A)"

💡 Terminologia:
- Classe Base / Superclasse / Pai
- Classe Derivada / Subclasse / Filha

# Sintaxe da Herança em C#

```
// CLASSE BASE (pai)
public class Pessoa
{
    public string Nome { get; set; }
    public int Idade { get; set; }

    public void Apresentar()
    {
        Console.WriteLine($"Olá, sou {Nome}");
    }
}

// CLASSE DERIVADA (filha) - usa : (dois pontos)
public class Funcionario : Pessoa  // Funcionario HERDA de Pessoa
{
    public decimal Salario { get; set; }
    public string Cargo { get; set; }

    public void Trabalhar()
    {
        Console.WriteLine($"{Nome} está trabalhando como {Cargo}");
    }
}

// Funcionario TEM: Nome, Idade (herdados) + Salario, Cargo
// Funcionario TEM: Apresentar() (herdado) + Trabalhar()
```

# Usando Herança na Prática

```
// Criando objetos
Funcionario func = new Funcionario();
func.Nome = "João";        // Herdado de Pessoa
func.Idade = 30;           // Herdado de Pessoa
func.Salario = 5000;       // Próprio de Funcionario
func.Cargo = "Desenvolvedor"; // Próprio de Funcionario

// Métodos herdados funcionam!
func.Apresentar();  // Herdado de Pessoa
func.Trabalhar();   // Próprio de Funcionario

// Output:
// Olá, sou João
// João está trabalhando como Desenvolvedor

// ✅ Reutilização: Funcionario não precisa redefinir
//    Nome, Idade, Apresentar() - já vêm da classe Pessoa!
```

# Palavra-chave base - Acessar Classe Pai

```
public class Pessoa
{
    public string Nome { get; set; }

    public Pessoa(string nome)
    {
        Nome = nome;
        Console.WriteLine("Construtor Pessoa chamado");
    }
}

public class Funcionario : Pessoa
{
    public decimal Salario { get; set; }

    // base() chama construtor da classe pai
    public Funcionario(string nome, decimal salario) : base(nome)
    {
        Salario = salario;
        Console.WriteLine("Construtor Funcionario chamado");
    }
}

// Uso:
var f = new Funcionario("João", 5000);
// Output:
// Construtor Pessoa chamado
// Construtor Funcionario chamado
```

# Construtores em Hierarquias de Herança

⚠️ ORDEM DE EXECUÇÃO:

1️⃣ Construtor da classe BASE (pai) executa PRIMEIRO
2️⃣ Construtor da classe DERIVADA (filha) executa DEPOIS

📌 Regra:

- Se classe pai tem construtor com parâmetros, classe filha DEVE chamar base()
- Usar : base(parâmetros) após construtor

💡 Analogia:

Construir casa: primeiro a fundação (base), depois as paredes (derivada)

# sealed - Impedir Herança

```
// sealed = IMPEDIR que outras classes herdem
public sealed class ContaBancaria
{
    public decimal Saldo { get; private set; }

    public void Depositar(decimal valor)
    {
        Saldo += valor;
    }
}

// ❌ ERRO: Não pode herdar de classe sealed!
public class ContaPoupanca : ContaBancaria  // ERRO DE COMPILAÇÃO!
{
    // Não permitido!
}

// 🎯 Quando usar sealed:
// • Segurança (impedir alteração de comportamento)
// • Performance (compilador otimiza)
// • Quando classe não foi projetada para herança

// Exemplo real: string é sealed em C#!
```

# Polimorfismo

Múltiplas Formas

# Polimorfismo - Conceito Fundamental

📌 Definição:

"Múltiplas formas" - Objetos de diferentes classes respondem de forma diferente à MESMA chamada

🎯 Tipos:

- Polimorfismo de SOBRESCRITA (override)
- Polimorfismo de SOBRECARGA (overload)

💡 Benefícios:

- Flexibilidade
- Extensibilidade
- Código mais genérico
- Facilita manutenção

🔑 Chave: virtual, override, abstract

# virtual e override - Sobrescrita de Métodos

```
public class Animal
{
    // virtual = pode ser sobrescrito
    public virtual void EmitirSom()
    {
        Console.WriteLine("Som genérico de animal");
    }
}

public class Cachorro : Animal
{
    // override = sobrescreve método da classe pai
    public override void EmitirSom()
    {
        Console.WriteLine("Au au!");
    }
}

public class Gato : Animal
{
    public override void EmitirSom()
    {
        Console.WriteLine("Miau!");
    }
}

// Uso polimórfico:
Animal a1 = new Cachorro();
Animal a2 = new Gato();
a1.EmitirSom();  // Au au!
a2.EmitirSom();  // Miau!
// Mesmo método, comportamentos diferentes!
```

# new vs override - Diferença CRÍTICA

```
// OVERRIDE - Polimórfico ✅
class Base
{
    public virtual void Metodo()
    { /*...*/ }
}

class Derivada : Base
{
    public override void Metodo()
    { /*...*/ }
}

Base obj = new Derivada();
obj.Metodo();
// Chama versão Derivada! ✅
```

```
// NEW - Oculta método ❌
class Base
{
    public void Metodo()
    { /*...*/ }
}

class Derivada : Base
{
    public new void Metodo()
    { /*...*/ }
}

Base obj = new Derivada();
obj.Metodo();
// Chama versão Base! ❌
// Não é polimórfico!
```

# Polimorfismo em Tempo de Execução

```
public class Forma
{
    public virtual double CalcularArea()
    {
        return 0;
    }
}

public class Circulo : Forma
{
    public double Raio { get; set; }
    public override double CalcularArea()
    {
        return Math.PI * Raio * Raio;
    }
}

public class Retangulo : Forma
{
    public double Largura { get; set; }
    public double Altura { get; set; }
    public override double CalcularArea()
    {
        return Largura * Altura;
    }
}

// Polimorfismo:
List<Forma> formas = new List<Forma>
{
    new Circulo { Raio = 5 },
    new Retangulo { Largura = 4, Altura = 3 }
};

foreach (var forma in formas)
{
    Console.WriteLine(forma.CalcularArea());  // Chama versão correta!
}
```

# Classes Abstratas

abstract class

# Classes Abstratas - Conceito

📌 Definição:

Classe que NÃO pode ser instanciada diretamente

Serve apenas como BASE para outras classes

🎯 Características:

- Palavra-chave: abstract
- Não pode usar new (não instancia)
- Pode ter métodos abstratos E concretos
- Pode ter construtores (chamados pelas filhas)
- Pode ter propriedades, campos

💡 Quando usar:

Quando há comportamento COMUM mas INCOMPLETO

que precisa ser definido pelas classes filhas

# Sintaxe de Classe Abstrata

```
// Classe abstrata - não pode instanciar
public abstract class Forma
{
    public string Cor { get; set; }  // Propriedade concreta

    // Construtor (chamado pelas filhas)
    public Forma(string cor)
    {
        Cor = cor;
    }

    // Método abstrato - SEM implementação
    public abstract double CalcularArea();
    public abstract double CalcularPerimetro();

    // Método concreto - COM implementação
    public void Desenhar()
    {
        Console.WriteLine($"Desenhando forma {Cor}");
    }
}

// ❌ ERRO: não pode instanciar!
Forma f = new Forma("Azul");  // ERRO DE COMPILAÇÃO!
```

# Métodos Abstratos vs Concretos

```
public abstract class Animal
{
    // Método ABSTRATO - sem corpo, termina com ;
    // Classe filha OBRIGATORIAMENTE implementa
    public abstract void EmitirSom();
    public abstract void Mover();

    // Método CONCRETO - com corpo (implementação)
    // Classe filha pode usar ou sobrescrever
    public virtual void Dormir()
    {
        Console.WriteLine("Zzzzz...");
    }

    public void Respirar()  // Concreto sem virtual
    {
        Console.WriteLine("Respirando...");
    }
}

// ⚠️ Regra:
// • Método abstract OBRIGA override na filha
// • Método virtual PERMITE override (opcional)
// • Método comum não pode ser sobrescrito
```

# Implementando Classe Abstrata

```
public abstract class Forma
{
    public abstract double CalcularArea();
}

// Classe CONCRETA - implementa métodos abstratos
public class Circulo : Forma
{
    public double Raio { get; set; }

    // OBRIGATÓRIO implementar métodos abstratos
    public override double CalcularArea()
    {
        return Math.PI * Raio * Raio;
    }
}

public class Quadrado : Forma
{
    public double Lado { get; set; }

    public override double CalcularArea()
    {
        return Lado * Lado;
    }
}

// Uso polimórfico:
Forma f1 = new Circulo { Raio = 5 };
Forma f2 = new Quadrado { Lado = 4 };
Console.WriteLine(f1.CalcularArea());  // 78.54
Console.WriteLine(f2.CalcularArea());  // 16
```

# Interfaces

Contratos de Comportamento

# Interfaces - Conceito

📌 Definição:

Contrato que define O QUE uma classe deve fazer sem definir COMO (sem implementação)

🎯 Características:

- Palavra-chave: interface
- Apenas assinaturas (sem corpo)
- Classe pode implementar MÚLTIPLAS interfaces
- Nomenclatura: começa com I (INotificador)
- Tudo é public por padrão

💡 Analogia:

Interface = Contrato de trabalho
(define obrigações, não como cumprir)

# Sintaxe de Interface

```
// Definindo interface
public interface INotificador
{
    // Apenas assinaturas (sem corpo)
    void EnviarNotificacao(string mensagem);
    bool ValidarDestinatario(string destinatario);

    // Propriedades (só get/set, sem implementação)
    string TipoNotificacao { get; }
}

// Implementando interface
public class EmailNotificador : INotificador
{
    public string TipoNotificacao => "Email";

    public void EnviarNotificacao(string mensagem)
    {
        Console.WriteLine($"Enviando email: {mensagem}");
    }

    public bool ValidarDestinatario(string destinatario)
    {
        return destinatario.Contains("@");
    }
}

// ✅ Classe DEVE implementar TODOS os membros da interface
```

# Múltiplas Interfaces - Diferencial do C#

```
// Várias interfaces
public interface IRepositorio
{
    void Salvar();
    void Excluir();
}

public interface INotificavel
{
    void Notificar(string msg);
}

public interface IValidavel
{
    bool Validar();
}

// Classe implementa MÚLTIPLAS interfaces (separadas por vírgula)
public class Cliente : IRepositorio, INotificavel, IValidavel
{
    public void Salvar() { /*...*/ }
    public void Excluir() { /*...*/ }
    public void Notificar(string msg) { /*...*/ }
    public bool Validar() { /*...*/ }
}

// ⚠️ Herança múltipla de CLASSES não é permitida em C#,
//    mas INTERFACES podem ser múltiplas!🎉
```

# Classe Abstrata vs Interface - Comparação

| Característica | Classe Abstrata | Interface |
|---|---|---|
| Palavra-chave | abstract class | interface |
| Implementação | Pode ter (concretos) | Não tem (só assinaturas) |
| Herança múltipla | ❌ Não (só uma) | ✅ Sim (várias) |
| Construtores | ✅ Pode ter | ❌ Não tem |
| Campos | ✅ Pode ter | ❌ Não tem |
| Modificadores | public, protected, etc | Tudo public |
| Quando usar | Relação "é um" | Relação "pode fazer" |
| Exemplo | Animal → Cachorro | IVoador → Aviao, Passaro |

# Casting e Type Checking

is, as, pattern matching

# Casting - Conversão de Tipos

```
// Hierarquia: Pessoa (pai) → Funcionario (filho)

// 1. UPCAST (implícito) - filho para pai
Funcionario func = new Funcionario();
Pessoa pessoa = func;  // OK automático! Funcionario É UMA Pessoa

// 2. DOWNCAST (explícito) - pai para filho
Pessoa p = new Funcionario();
Funcionario f = (Funcionario)p;  // Cast explícito necessário

// ⚠️ PERIGO: Downcast pode falhar!
Pessoa p2 = new Pessoa();  // Pessoa pura, não é Funcionario
Funcionario f2 = (Funcionario)p2;  // ❌ EXCEPTION em runtime!

// ✅ SOLUÇÃO: Verificar ANTES de fazer cast
if (p2 is Funcionario)
{
    Funcionario f3 = (Funcionario)p2;
}
else
{
    Console.WriteLine("Não é Funcionario!");
}
```

# Operador is - Verificar Tipo

```
// is verifica se objeto é de determinado tipo

Animal animal = new Cachorro();

// Forma tradicional
if (animal is Cachorro)
{
    Console.WriteLine("É um cachorro!");
    Cachorro c = (Cachorro)animal;
    c.Latir();
}

// Forma moderna - Pattern Matching (C# 7+)
if (animal is Cachorro cachorro)
{
    // Já faz cast e cria variável 'cachorro'
    cachorro.Latir();  // Pode usar direto!
}

// Verificar vários tipos
if (animal is Cachorro) { /*...*/ }
else if (animal is Gato) { /*...*/ }
else if (animal is Passaro) { /*...*/ }

// is retorna true/false
bool eCachorro = (animal is Cachorro);
```

# Operador as - Cast Seguro

```
// as faz cast SEGURO - retorna null se falhar

Pessoa pessoa = new Pessoa();

// Forma insegura (pode lançar exception)
Funcionario f1 = (Funcionario)pessoa;  // ❌ EXCEPTION!

// Forma SEGURA com 'as'
Funcionario f2 = pessoa as Funcionario;  // Retorna null se falhar

if (f2 != null)
{
    Console.WriteLine(f2.Salario);
}
else
{
    Console.WriteLine("Não é Funcionario");
}

// Uso comum:
Animal animal = GetAnimal();
Cachorro cachorro = animal as Cachorro;
if (cachorro != null)
{
    cachorro.Latir();
}

// ⚠️ as só funciona com tipos de referência (classes)
```

# Pattern Matching - Recurso Moderno

```
// Switch com Pattern Matching (C# 8+)

public string DescreverForma(Forma forma)
{
    return forma switch
    {
        Circulo c => $"Círculo com raio {c.Raio}",
        Retangulo r => $"Retângulo {r.Largura}x{r.Altura}",
        Triangulo t => $"Triângulo de área {t.CalcularArea()}",
        _ => "Forma desconhecida"
    };
}

// Property Pattern
public decimal CalcularDesconto(Produto produto)
{
    return produto switch
    {
        { Preco: > 1000 } => produto.Preco * 0.10m,  // 10% se > 1000
        { Preco: > 500 } => produto.Preco * 0.05m,   // 5% se > 500
        { Preco: > 100 } => produto.Preco * 0.02m,   // 2% se > 100
        _ => 0
    };
}

// Muito mais expressivo que if/else!
```

# Interface Segregation - Preview SOLID

📌 Princípio:

"Clientes não devem ser forçados a depender de interfaces que não usam"

❌ Interface grande (ruim):

```
interface ITrabalhar { Trabalhar(), Descansar(),
Comer(), Dormir(), Programar(), Gerenciar() }
```

✅ Interfaces pequenas (bom):

```
interface ITrabalhar { Trabalhar() }
interface IDescansar { Descansar() }
interface IProgramar { Programar() }
```

💡 Benefícios:

- Classes só implementam o necessário
- Mais flexibilidade
- Fácil manutenção

# Hora de Praticar!

Exercícios Práticos

# Exercício 1: Hierarquia de Classes

🎯 DESAFIO:
Criar hierarquia Pessoa → Cliente, Funcionário

📋 Requisitos:

1️⃣ Classe Abstrata Pessoa
- Nome, CPF, DataNascimento
- Método abstrato: Apresentar()
- Método concreto: CalcularIdade()

2️⃣ Classe Cliente (herda Pessoa)
- CreditoDisponivel
- override Apresentar(): mostrar como cliente
- Método: RealizarCompra(valor)

3️⃣ Classe Funcionario (herda Pessoa)
- Salario, Cargo, DataAdmissao
- override Apresentar(): mostrar como funcionário
- Método: CalcularTempoEmpresa()

# Fim de Jogo

https://w.app/gil

Obrigado e até a próxima aula!

# Obrigado!

Dúvidas?

prof.geovani@fiel.edu.br